

LINEAR QUEUES

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 5

struct queue {
    int a[MAX];
    int front;
    int rear;
};

typedef struct queue qu;

void createEmptyQueue(qu *q) {
    q->front = -1;
    q->rear = -1;
}

int isfull(qu *q) {
    if (q->rear == (MAX - 1))
        return 1;
    else
        return 0;
}

int isempty(qu *q) {
    if (q->front == -1 || q->front > q->rear)
        return 1;
    else
        return 0;
}

void enqueue(int val, qu *q) {
```

```

if (isfull(q))
    printf("Queue full\n");
else {
    if (q->front == -1)
        q->front = 0;
    q->rear++;
    q->a[q->rear] = val;
    printf("Enqueued: %d\n", val);
}
}

```

```

int dequeue(qu *q) {
    if (isempty(q)) {
        printf("Queue empty\n");
        return -1;
    }
    else {
        int temp = q->a[q->front];
        q->front++;
        return temp;
    }
}

```

```

int main() {
    printf("Name: Niveditha A\nReg No: 24BCE2000\n");
    qu *q = (qu *)malloc(sizeof(qu)) ;
    createEmptyQueue(q);
    enqueue(10, q);
    printf("One element pushed into the queue \n");
    printf("Front = %d, Rear = %d \n", q->front, q->rear);
    enqueue(20, q);
    enqueue(30, q);
}

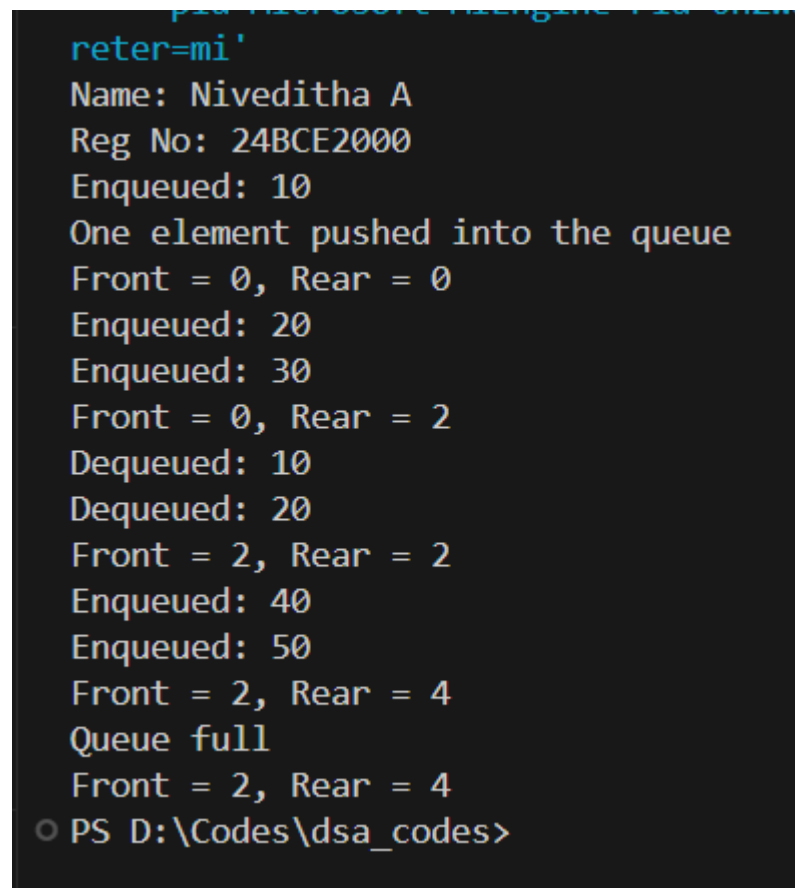
```

Niveditha A - 24BCE2000

```

printf("Front = %d, Rear = %d \n", q->front, q->rear);
printf("Dequeued: %d\n", dequeue(q));
printf("Dequeued: %d\n", dequeue(q));
printf("Front = %d, Rear = %d \n", q->front, q->rear);
enqueue(40, q);
enqueue(50, q);
printf("Front = %d, Rear = %d \n", q->front, q->rear);
enqueue(60, q);
printf("Front = %d, Rear = %d \n", q->front, q->rear);
return 0;
}

```



```

reter=mi'
Name: Niveditha A
Reg No: 24BCE2000
Enqueued: 10
One element pushed into the queue
Front = 0, Rear = 0
Enqueued: 20
Enqueued: 30
Front = 0, Rear = 2
Dequeued: 10
Dequeued: 20
Front = 2, Rear = 2
Enqueued: 40
Enqueued: 50
Front = 2, Rear = 4
Queue full
Front = 2, Rear = 4
PS D:\Codes\dsa_codes>

```

CIRCULAR QUEUES

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 5

struct queue {
    int a[MAX];
    int front;
    int rear;
};
typedef struct queue qu;

void createEmptyQueue(qu *q) {
    q->front = -1;
    q->rear = -1;
}

int isfull(qu *q) {
    if ((q->rear + 1) % MAX == q->front)
        return 1;
    else
        return 0;
}

int isempty(qu *q) {
    if (q->front == -1)
        return 1;
    else
        return 0;
}
```

```

void enqueue(int val, qu *q) {
    if (isfull(q))
        printf("Queue full\n");
    else {
        if (q->front == -1)
            q->front = 0;
        q->rear = (q->rear + 1) % MAX;
        q->a[q->rear] = val;
        printf("Enqueued: %d\n", val);
    }
}

```

```

int dequeue(qu *q) {
    if (isempty(q)) {
        printf("Queue empty\n");
        return -1;
    }
    int temp = q->a[q->front];
    if (q->front == q->rear) {
        // Queue becomes empty
        q->front = -1;
        q->rear = -1;
    } else {
        q->front = (q->front + 1) % MAX;
    }
    return temp;
}

```

```

int main() {
    printf("Name:Niveditha A \nReg No: 24BCE2000 \n");
    qu *q = malloc(sizeof(qu));
    createEmptyQueue(q);
}

```

Niveditha A - 24BCE2000

```

enqueue(10, q);
printf("First element added \n");
printf("Front = %d, Rear = %d \n", q->front, q->rear);
enqueue(20, q);
enqueue(30, q);
enqueue(40, q);
enqueue(50, q);
printf("Front = %d, Rear = %d \n", q->front, q->rear);
enqueue(60, q);
printf("Dequeued: %d\n", dequeue(q));
printf("Dequeued: %d\n", dequeue(q));
printf("Front = %d, Rear = %d \n", q->front, q->rear);
enqueue(70, q);
printf("Front = %d, Rear = %d \n", q->front, q->rear);
enqueue(80, q);
printf("Front = %d, Rear = %d \n", q->front, q->rear);
printf("Dequeued: %d\n", dequeue(q));
printf("Dequeued: %d\n", dequeue(q));
printf("Dequeued: %d\n", dequeue(q));
printf("Dequeued: %d\n", dequeue(q));
printf("Front = %d, Rear = %d \n", q->front, q->rear);
printf("Dequeued: %d\n", dequeue(q));
printf("Front = %d, Rear = %d \n", q->front, q->rear);
return 0;
}

```

```
reter=mi'  
Name:Niveditha A  
Reg No: 24BCE2000  
Enqueued: 10  
First element added  
Front = 0, Rear = 0  
Enqueued: 20  
Enqueued: 30  
Enqueued: 40  
Enqueued: 50  
Front = 0, Rear = 4  
Queue full  
Dequeued: 10  
Dequeued: 20  
Front = 2, Rear = 4  
Enqueued: 70  
Front = 2, Rear = 0  
Enqueued: 80  
Front = 2, Rear = 1  
Dequeued: 30  
Dequeued: 40  
Dequeued: 50  
Dequeued: 70  
Front = 1, Rear = 1  
Dequeued: 80  
Front = -1, Rear = -1  
PS D:\Codes\dsa_codes>
```